SDP



Module 6: Custom Agents

⏰ Turn capabilities into roles

"Agents are where instructions, prompts, skills, and MCP access become an actual workflow teammate."

M1 Instructions › M2 Run Mode › M3 Prompts › M4 Skills › M5 MCP › **M6 Agents**

⏰ : Monday, 3:45 PM

Sarah stares at the `.github` folder. Prompts, skills, an MCP server, a universe file. Five modules of work. She still has to wire all of it together herself, every time.

> " Five modules of work sitting in this repo. Right now I have to invoke all of it manually, every time. What if there was just... an agent for that?"

Module 1
Document the standards
Instructions created a reliable baseline.

Module 2
Plan the work
Plan mode made complex work reviewable.

Module 3
Save the best prompts
Repeated prompts became reusable entry points.

Module 4
Package expertise
Specialized expertise became callable skills.

Module 5
Connect live systems
MCP brought real databases and APIs into the loop.

Module 6
Bundle into agent
Now the goal is actual delivery.

⚡ You *could* start with agents — but agents are only as good as what they run. Skills need a tested prompt. Agents need skills and live data. The sequence is the architecture.

The Team, Before Module 6

⚠ WRONG ANSWERS ONLY

Jessica
The Eager Junior

"Custom agents are just Copilot with a different name. Like a theme. A personality skin."

About to discover it's considerably more than a skin.

David
The Seasoned Architect

"A custom agent is a system prompt with a YAML header. I could reproduce this in ten minutes. I'll wait."

He could write the YAML. He could not write the agent.

Sarah
The Skeptical Senior

"Once it's built, the agent runs on its own indefinitely. Zero maintenance. That's the pitch, right?"

"Zero maintenance" has never been a pitch that held.

Elena
The Quality Champion

"The agent validates its own output. Self-testing. You don't need to review it — it reviews itself."

She will spend the next hour testing that assumption to destruction.

Marcus
The DevOps Developer

"Custom agents replace Copilot entirely. You're basically writing your own model config. That sounds like a lot."

Half-checked-out. Catastrophically misinformed.

Rafael
The Product Visionary

"So I assign the Jira ticket to the agent, add it to the sprint, and check out for the day. That's what this is, right?"

Already blocking off his afternoon calendar.

Built-In Agents You Already Have

Copilot Cloud Agent

Assigns itself to a GitHub Issue, works autonomously in a sandboxed environment, and opens a pull request when done.

- Triggered from a GitHub Issue — assign Copilot as the owner
- Reads repo context, plans, implements, and runs tests in a secure cloud workspace
- Delivers a PR with a summary of decisions made

Uses your workshop work: respects repo instructions, invokes skills, and queries MCP servers — everything from Modules 1–5 shapes how it works.

Best for: well-scoped tasks with a clear issue description

Copilot Code Review

Reviews pull requests automatically and leaves inline comments with actionable suggestions — no setup required.

- Request a review from `@github-copilot` on any PR
- Surfaces logic issues, missing edge cases, and style inconsistencies
- Can be configured with custom review instructions in repo settings

Uses your workshop work: respects repo instructions and invokes skills — your Module 1 and 4 work directly improves review quality.

Best for: a fast first-pass review before human reviewers engage

These are GitHub-hosted agents. Module 6 is about your own — purpose-built for the FanHub workflow.

◆◆ What You'll Build

6.1 15 min · All ⭐

Read the Plan Agent

Trace `plan.agent.md` as architecture — why every tool is listed, what's deliberately absent, and how the Explore subagent and handoff chain work together.

6.2 30 min · Marcus ⭐

Explore Agent Teams

Install Squad, initialize it in the repo, and watch a natural language command cast a named team, hold a design review, and commit decisions — all without manual wiring.

6.3 25 min · All ⭐🤝 capstone

Build `@content-author`

Wire M1 context, M3/M4 skills, and M5 live data into one agent. One instruction scaffolds an entity end-to-end — and catches accuracy errors before migration.

Study the pattern → see where it scales → build it yourself.

Module 6 operationalizes everything the workshop has taught so far.

What Custom Agents Actually Are

A role-specific operating mode

- YAML frontmatter defines tools, model, handoffs, and related controls
- Markdown instructions define the workflow and constraints
- The agent appears as a reusable preset for a specific kind of work

A way to encode workflow boundaries

- Review agents can be read-only
- Implementation agents can have edit tools
- Handoffs connect roles without collapsing their boundaries

A place where earlier customizations meet

- Repository instructions still apply underneath
- Skills can provide domain knowledge on demand
- MCP tools can expand what the agent can do

The real payoff

You stop re-explaining the mode of work and start selecting the right teammate for the job.

💭 *Marcus: "Wait — sorry, I was half on something else — but the tool list is the role boundary? That's not configuration. That's a job description that enforces itself."*

Agents vs Instructions vs Skills vs MCP

Instructions

Tell Copilot how to behave.

Prompts

Package repeated requests.

Skills

Teach a specialized capability.

Agents

Compose the right tools and workflows into a role.

Agents are the orchestration layer.

They decide which capabilities are available, when to stop, and where the work flows next.

> 💭 *Jessica: "Oh. OH. So when it kept going rogue and doing whatever it wanted — that wasn't a Copilot problem. That was a me-not-knowing-there-were-four-different-things problem. Those are very different problems."*

Core Agent Design Patterns

Tool selection is policy

- Read-only tools enforce research or review boundaries
- Editing tools appear only where implementation is expected
- The tool list shapes the behavior as much as the prompt does

Instructions define the workflow

- Stopping rules prevent scope creep
- Style guides make output more consistent
- Ordered phases reduce improvisation

Handoffs

Turn isolated tasks into guided workflows from one role to the next.

Model and scope controls

Pick the environment and invocation pattern that fit the job.

Subagent delegation

Isolate focused subtasks — research, analysis, parallel reviews — in separate context windows. The main agent gets only the summary.

💭 ***Rafael:*** *"Defined role, explicit stopping point, structured handoff. I've been asking engineering for that for years. Turns out it just needed to be a config file."*

The AGENTS.md Standard

What it is

An open community convention for documenting AI agent instructions in a way that works across editors — VS Code, Cursor, and others. A plain markdown file in `.github/agents/` that any agent-aware tool can read.

YAML frontmatter + Markdown body

```
---
name: character-review
description: Review character changes
tools: ['search', 'fetch']
handoffs:
  - agent: implement
---

# Workflow instructions here
Analysis only. Do not modify files.
```

Why it matters for the team

- **Portable** — The same `.agent.md` file works in any AGENTS.md-compatible editor
- **Shareable** — Commit agent configs to the repo; the whole team gets the same presets
- **Discoverable** — New team members open VS Code and the agents are already there
- **Convention-based** — No proprietary lock-in; the format is community-owned

Where agents live

```
.github/agents/
  character-review.agent.md
  implement.agent.md
  breaking-bad-expert.agent.md
~/.config/github-copilot/agents/
```

👉 agents.md — The open standard spec: agents.md

❌ Before vs ✨ After

❌ Before

Work starts with reconfiguration

The user re-explains whether this is planning, implementation, review, or analysis work.

Boundaries are easy to blur

A planning conversation can drift into editing or a review session can accidentally make changes.

Workflow transitions are manual

Even when the next step is obvious, the handoff from plan to implement or implement to review is not encoded.

✨ After

Roles become instant presets

The team picks the right agent and starts in the right mode immediately.

Constraints are enforced, not implied

Read-only planning or review is backed by tool restrictions and instructions, not just good intentions.

Handoffs create a real workflow chain

Plan, implement, and review become a sequence the team can actually reuse.

What to Validate in the Workshop

Role fit

Does the agent's tool list and instruction set actually match the work it is supposed to perform?

Workflow reliability

Does the agent consistently stay in scope and produce the kind of output the role needs?

Handoff quality

Does the transition to the next role preserve context and reduce manual setup?

If it stays in its lane, delivers reliably, and hands off cleanly — you've built an agent. Everything else is just a prompt with a name.

> *"I built a checklist for validating agents. Then I realized I was describing the agent itself."*
>
> — Elena

How Subagents Work

THE 5-STEP FLOW

1. You (or agent instructions) describe a complex task
2. Main agent recognizes a subtask that benefits from isolated context
3. Spawns a subagent — passes only the **relevant subtask**, nothing else
4. Subagent works autonomously, returns only a **summary**
5. Main agent incorporates the result and **continues cleanly**

WHAT YOU SEE IN VS CODE

```
▶ Explore subagent                    Running ...
  Reading file: routes/locations.ts
  Searching codebase: Entity pattern
✓ Explore subagent                    Done
Returned: summary only — tool calls hidden
```

🔑 The key insight

Subagents are **agent-initiated**, not user-invoked. Enable `runSubagent` in the agent's tool list — then the agent decides when to use it.

> *"You're still on the 'how.' I'm already at 'what does this do to our sprint velocity.' The answer is: a lot."*
>
> — Rafael

Invocation Patterns

Research before acting

Before writing code, delegate research to a subagent. Main agent gets a clean recommendation — not a context dump.

```
# prompt
Research OAuth 2.0 patterns
for Node.js apps. Compare
against current impl and
return a recommendation.
```

Parallel analysis

Run multiple subagents simultaneously — each with a clean slate, no anchoring bias from other findings.

```
# prompt
Analyze in parallel:
1. Duplicate code
2. Unused exports
3. Security gaps
Compile into a plan.
```

Multi-perspective review

Specialized agents review the same code independently — no anchoring, each uses the right tools for its lens.

```
# prompt
Run in parallel:
- @security-reviewer
- @performance-reviewer
- @a11y-reviewer
Consolidate findings.
```

In all three patterns, the main agent's context stays **clean and focused** on the high-level task.

Coordinator + Worker Pattern

HOW IT'S STRUCTURED

```
feature-builder.agent.md
name: Feature Builder
tools: ['agent', 'edit', 'read']
agents: ['Planner', 'Implementer', 'Reviewer']
```

```
planner.agent.md (worker)
name: Planner
user-invocable: false
tools: ['read', 'search']
# no edit, no create — read-only by design
```

user-invocable: false — worker agents stay hidden from the agent dropdown. Only the coordinator can invoke them.

THE FLOW

Feature Builder receives the request — coordinates everything

↓ delegates to

Planner — read-only, breaks feature into tasks

Implementer — write access, executes tasks

Reviewer — read-only, checks the output

"Hold on — read-only Planner, write-access Implementer. That's least-privilege. For agents. I was half-checked-out and now I need to go rewrite some things."

— Marcus

🔨 Exercises — Agents

3 exercises · ~70 minutes · shared leadership

Study the pattern → explore what it becomes → build it yourself

6.1 — 15 min · All ⭐

Study the Default Plan Agent

Open `.github/agents/plan.agent.md` and read it as architecture. Trace why every tool is listed, what's deliberately absent, and how the handoff chain passes context forward.

You'll identify:

- Explore subagent pattern
- Tool list as role boundary
- Stopping rules + handoff chain

6.2 — 30 min · Marcus ⭐

Explore Agent Teams with Squad

Install Squad CLI and run both prompts. Watch it cast a named team, hold a design review, and commit decisions to the repo — all from a single natural-language instruction.

You'll see:

- Subagent pattern at scale
- Ceremony-aware coordination
- Decisions committed to the repo

6.3 — 25 min · All ⭐🤝 capstone

Build the @content-author Agent

Use the prompt on the next slide to generate `content-author.agent.md`, then invoke it to add a Locations entity end-to-end. One instruction activates five modules of investment.

You'll produce:

- A working agent config
- Full-stack entity + MCP tools
- Accuracy-verified seed data

EXERCISE

6.1 Study the Default Plan Agent

All ⭐ | 15 min

Before building a custom agent, study how the default `@plan` agent is designed — its Explore subagent, its tool list, its stopping rules. **This is where you learn to read agent configs like architecture, not just configuration files.**

STEPS

1. Open the Plan agent file — find `agents: ['Explore']` and explain why it spawns a subagent before doing anything else
2. Study the tool list — note what is *absent* (`edit`, `create`, `delete`). The only write tool is `vscode/memory` — for the plan itself, never code
3. Trace the handoff chain: `@plan` → `Start Implementation`. Name the design decision each step encodes

SUCCESS CRITERIA

- [ ] Can explain why Explore is a subagent, not a tool
- [ ] Can name 3 tools not on the list and why that's intentional
- [ ] Can trace the full handoff chain and explain each step

> 💬 *David: "Technically, a custom agent is a deliberate override of the default execution context. The interesting question isn't whether you can override it — it's whether you understand what you're overriding well enough to do it intentionally."*

Example: default plan agent -- key patterns

PLAN.AGENT.MD

```
---
name: Plan
description: Researches and outlines multi-step plans
target: vscode
disable-model-invocation: true
tools: ['search', 'read', 'web',
   'vscode/memory', 'agent',
   'vscode/askQuestions']
agents: ['Explore']
handoffs:
 - label: Start Implementation
   agent: agent
   prompt: Start implementation
   send: true
 - label: Open in Editor
   agent: agent
---

You are a PLANNING AGENT.

<rules>
 STOP if you consider running
 file editing tools.
```

① Explore subagent pattern

`agents: ['Explore']` — the agent launches Explore subagents via `#runSubagent` to gather parallel context *before* planning begins. Delegate research; stay focused on planning.

② Read-only tool list

No `edit`, `create`, or `delete` in the tool list. `vscode/memory` is the *only* write capability — for the plan file only. **What's absent is as deliberate as what's present.**

③ Handoff chain

Two handoff buttons appear: Start Implementation passes context to @implement; Open in Editor saves plan to a file. Context flows forward — no copy-paste.

`.github/agents/plan.agent.md` · study this before building your own

Marcus: "I've been trying to enforce that boundary with code review

EXERCISE

6.2 Explore Agent Teams with Squad

Marcus ⭐ | 15 min

Single custom agents solve focused workflow problems, but it is still hard to picture what a persistent multi-agent team could look like in practice. **This exercise installs Squad, initializes it inside the repo, and uses it from VS Code as an "art of the possible" capstone for agent teams.**

STEPS

1. Install `@bradygaster/squad-cli` and run `squad init`
2. Open Copilot Chat in VS Code and select the `Squad` agent
3. Try one FanHub-style team request and compare it to a single custom agent flow

SUCCESS CRITERIA

- [ ] Squad is installed and initialized in the repo
- [ ] You can explain the difference between a custom agent and an agent team
- [ ] Squad's team proposal includes a clear lead and specialists that match the request

💬 *Marcus: "Wait — so you can chain these? Like, one agent hands off to the next? I'm already mapping this to our deploy pipeline — actually no, finish the demo first. I'm coming back to that though."*

Example: Squad agent team setup

```
# Install Squad CLI
npm install -g @bradygaster/squad-cli

# Initialize Squad in the repo
squad init

# Open Copilot Chat and type:
@squad let's hire a team to work on the [language] version on FanHub

@squad let's have a meeting to figure out the best way to implement #file:FEATURE-CHARACTER-DETAIL.md
```

What Squad does

- Routes task to a **persistent multi-agent team**
- Specialized agents **collaborate in parallel**
- Output is **coordinated** — not a single chat response
- Team config lives in the repo — **everyone gets the same team**

vs. your handoff chain

Your `@plan → @implement` chain is explicit and deterministic — you designed each transition. Squad is autonomous — it decides how to route internally. Both are valid; the right choice depends on how much control you need.

Connection to Exercise 6.1

The plan agent uses `agents: ['Explore']` to spawn one subagent at a time. Squad extends that into a persistent, named team -- 6.1 shows the single-agent version; Squad shows where the pattern goes at scale.

Squad Output — "Hire a Team"

WHAT SQUAD DID

```
Read .squad/
Read .squad/casting
Read .squad/agents
Good — Ralph and Scribe already exist.
Creating Firefly team...
Created 4 todos
✓ Updated .squad/team.md
✓ Created casting/policy.json
✓ Created casting/registry.json
✓ Created casting/history.json
Creating agent charters & histories...
✓ agents/mal/charter.md
✓ agents/mal/history.md
✓ agents/kaylee/charter.md
✓ agents/kaylee/history.md
✓ agents/wash/charter.md
✓ agents/wash/history.md
✓ agents/zoe/charter.md
✓ agents/zoe/history.md
```

✅ TEAM HIRED

	Agent	Role
	Mal	Lead / Architect
	Kaylee	Backend Dev
	Wash	Frontend Dev
	Zoe	Tester
	Scribe	Silent logger
	Ralph	Work queue monitor

Each agent has their charter and history pre-seeded with codebase context — they can hit the ground running.

Try next:

Squad Output — "Have a Meeting"

WHAT SQUAD DID

```
Read .squad/ceremonies.md
Maps to: Design Review
Facilitator: Mal + Kaylee, Wash, Zoe
Read agents/mal/history.md
Read docs/FEATURE-CHARACTER-DETAIL.md
Read .squad/decisions.md
Read Backend/Models/ (5 files)
Read Frontend/Components/ (3 files)
✓ Created decisions/inbox/
✓ Created mal-character-detail-design-review.md
```

Mal's call on scope

Episodes deferred to Phase 2.

DESIGN REVIEW OUTPUT

✅ Phase 1 In	❌ Phase 2
Character identity + bio	Episode list
Quotes (full text)	Advanced filtering
Related characters	Server-side favorites
Favorite toggle (localStorage)	
Loading + error states	

Action items (work order: 1→2 ‖ 3→4 ‖ 5→6 ‖ 7)

Kaylee: Add Episode nav to Quote model → implement GET /api/characters/{id} DTO

Wash: CharacterDetailDto → CharacterDetail.razor → update Characters.razor

Zoe: Integration tests + regression test (confirm 50-char bug stays broken)

Try next:

Squad Decision — Character Detail Design Review

Copy

```
# Decision: Character Detail Page — Design Review

**Decided by:** Mal (Lead/Architect)
**Date:** 2026-03-31
**Participants:** Kaylee (Backend), Wash (Frontend), Zoe (Testing)
**Related feature:** `docs/FEATURE-CHARACTER-DETAIL.md`

---

## 1. Episode List Phase Decision

**DECISION: Episode list is DEFERRED to Phase 2.**

The spec lists episodes as Phase 1 scope, but the codebase has no episode-character junction table. `Episode.Characters` is
marked `[NotMapped]`. Phase 1 will deliver character identity + quotes + relationships + favorites toggle instead.

The spec's out-of-scope quote section is swapped IN because the data model already supports it via `Quote.CharacterId`, and the
character detail endpoint bypasses the `GET /api/quotes` truncation bug (intentional — must NOT be touched).

**Phase 1 scope (revised):**

- OK Character identity (name, bio, image, actor, status, tagline, characterType)
- OK Quotes (via character detail endpoint — full text, no truncation bug)
- OK Related characters (with relationship type and resolved names)
- OK Favorite toggle (localStorage)
- OK Loading and error states
- NO Episode list (deferred to Phase 2 — requires migration)
```

6.3

EXERCISE · CAPSTONE ⭐ Five Modules, One Agent

Build the @content-author Agent

All ⭐🤝 | 25 min

Every prior module built a reusable piece. **This capstone wires them together: one agent, one instruction — "Add a Quotes entity to FanHub with 5 iconic lines" — and every module's investment pays off in a single session.**

📋 STEPS

1. Create `.github/agents/content-author.agent.md` with full editing tools, `agents: ['Explore']`, and the fanhub-db MCP server
2. Write instructions: check universe doc, query MCP, invoke `create-card-and-page skill`, run `check-data-accuracy` on every seeded record before migration
3. Run: *"Add a Quotes entity to FanHub — seed it with 5 iconic Breaking Bad lines"* and let the agent work
4. Debrief: for each thing the agent did, name the module that made it possible

✅ SUCCESS CRITERIA

- ☐ Quote entity scaffolded end-to-end (model → controller → Blazor page)
- ☐ The `check-data-accuracy` skill catches the planted misattribution before migration
- ☐ Every participant names the module behind each action the agent took

💭 ***Sarah:*** *"One instruction. It scaffolded the stack, queried live data, validated against the universe doc, and caught a misattribution before migration. I was keeping count. That's five manual steps I didn't do. Okay — that's a real number."*

Creating the content-author Agent

START WITH THIS PROMPT

I want to create a new agent. I want to be able to call it like this:

"Add a Locations entity to FanHub and seed it with 5 iconic Breaking Bad locations"

I want the agent to:

- Use the `create-card-and-page` skill to do the scaffolding work
- Use the `check-data-accuracy` skill to verify the seed data against the universe file
- Use the built-in `Explore` agent to check for architecture and code patterns before starting
- Update `fanhub-api-server.js` with the new entity endpoints after a successful build, so the MCP server exposes them automatically

Can you help me create the `.github/agents/content-author.agent.md` file?

WHAT COPILOT WIRES TOGETHER

- `agents: ['Explore']` — reads the codebase before acting, same pattern as the Plan agent
- `create-card-and-page` — skill from M4 does the scaffolding; the agent delegates, doesn't reimplement
- `check-data-accuracy` — skill from M3/M4 validates seed data against the universe file before anything ships
- `fanhub-api-server.js` — agent adds entity endpoints post-build so the MCP server exposes them to future skills

💡 You don't need to know the agent format.

`.github/agents/`

Example: scaffold-entity.agent.md -- the capstone config

Copy

SCAFFOLD-ENTITY.AGENT.MD

```
---
name: scaffold-entity
description: >
  Scaffolds a new entity type (e.g. Quotes, Locations,
Factions) into FanHub
  with a full-stack implementation: backend model, DbContext
registration, seed data,
  EF Core migration, REST controller, frontend model, homepage
stat card + nav-card,
  header navigation link, and a Blazor list page with an inline
add form. After
  scaffolding, verifies seed data accuracy against canonical
Breaking Bad sources.
  Trigger phrases: "add a [Entity] entity", "scaffold
[Entity]", "add [Entity] to FanHub",
  "create a [Entity] page", "add [Entity] with seed data".
tools:
  - search/codebase
  - search/usages
  - search/fileSearch
  - search/textSearch
  - search/listDirectory
  - read/readFile
  - read/problems
  - edit/editFiles
  - edit/createFile
```

① Explore before acting

`agents: ['Explore']` — same pattern as the Plan agent. Recon first, scaffold second. Architecture knowledge informs every naming and structural decision.

② Delegates to skills, doesn't reimplement

Phases 3 and 4 load `create-card-and-page` and `check-data-accuracy` by file path. The agent orchestrates; the skills do the heavy lifting.

③ Full write access + run terminal

Unlike the Plan agent, this one *does* edit and create files, run migrations, build, and start the server. Tool list matches the role.

④ MCP loop closes the circle

After scaffolding, it updates `fanhub-api-server.js` so the new entity is immediately reachable via MCP tools — future

✅ Module Checklist

By the end of Module 6, the team understands how agents are designed — and has built one.

🧠 Study the pattern

- Explore subagent pattern — parallel research before planning
- Tool list as role boundary — what's absent is intentional
- Handoff chain: @plan → implementation

🚀 Explore Squad

- The Explore pattern at team scale
- Persistent multi-agent coordination in VS Code
- Difference between a workflow preset and an agent team

⭐ Build the capstone

EXERCISE 6.2

- @content-author wires M1–M5 in one agent
- Quotes entity scaffolded + validated in one instruction
- Every participant names the module behind each action

What good looks like

The team can read an agent config like architecture — and can write one that composes everything they've built since Module 1.

Why this matters

Module 6 is where earlier investments — instructions, skills, MCP — stop feeling like separate tools and start working as a coordinated system.

SDP



Module 6 Is Locked In

🌐 Congratulations!

🧠

You Studied the Pattern

Explore subagents, tool boundaries, stopping rules, and handoff chains — read as architecture

🚀

You Explored the Horizon

Explore subagents, tool boundaries, stopping rules, and handoff chains — read as architecture

⭐

You Built the Capstone

Brownfield author activated five modules of investment from a single instruction

Module 6 is where the earlier customizations stop feeling like parts and start feeling like an operating model.

The team built something that compounds across every sprint from here.

Reflect & Discuss

STORY MOMENTS

Marcus saw one agent hand off to the next and immediately mapped it to the deploy pipeline — a mental model shift in real time

Sarah's five-module payoff: "checked the canon, checked the live data, scaffolded the whole stack, caught an attribution error before migration" — all from one instruction

Marcus gave the agent a bad question on purpose and it said "I don't know" — the first time he trusted it

ONE BIG QUESTION

If you could design one agent for your team today — something that owns a whole category of work end-to-end — what would it do, and what would it need to know?

The Team, After Module 6

✓ AFTER THE MODULE

Jessica

The Eager Junior

"I built an agent today. A real one. It has a name, instructions, and it knows our entire show. It is not a skin."

Built something with her name on it. Corrected herself without being asked.

David

The Seasoned Architect

"I said I could write the YAML in ten minutes. I was wrong about what I was writing."

Conceding the diff between the config and the collaborator.

Sarah

The Skeptical Senior

"The content-author agent scaffolded the whole stack and caught an attribution error before migration. First pass. That's what five modules feels like when it's working."

Better first draft than three modules of iteration.

Elena

The Quality Champion

"It has constraints, a persona, and escalation rules. That's the opposite of self-validating. That's exactly why I trust it."

Trusted it because it doesn't trust itself.

Marcus

The DevOps Developer

"I gave it a bad question on purpose. Thought I was writing a model config. Turns out I was writing a teammate."

Snapped into focus. Didn't see that coming.

Rafael

The Product Visionary

"I didn't get to check out. But one deployment covers every future content task for this feature. That's not checking out — that's leverage."

Afternoon calendar unblocked. For different reasons than expected.